

Lab 1 - Requirements Engineering & UML Use-Case Modelling

Problem Statement #26: Peer-to-Peer Carpooling & Expense Splitter

Name: Kanchhit | PES University

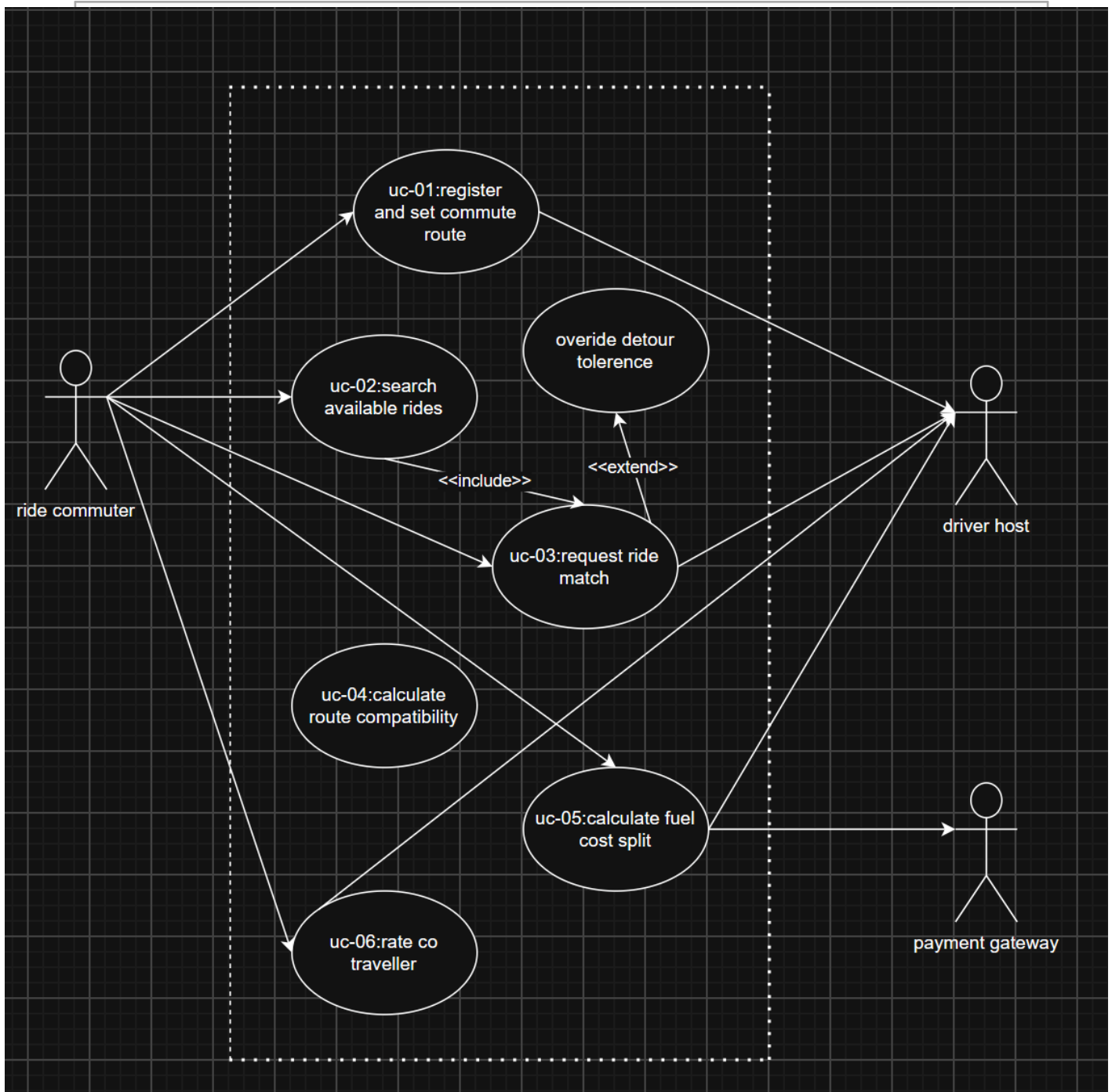
1. Requirements Table

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall calculate route compatibility between driver and passenger commute paths, permitting matches only within a 1.5 km detour tolerance.	High	Pass: Match confirmed within tolerance. Fail: Detour exceeding 1.5 km accepted.	Core matching functionality
FR-002	Functional	The system shall allow a user to register and set their daily commute route with origin, destination, and departure time.	High	Pass: Route saved and visible on profile. Fail: Route data missing or not saved.	Needed before any matching can happen
FR-003	Functional	The system shall let a rider search available rides and show compatible drivers sorted by detour distance.	High	Pass: Results returned sorted by detour. Fail: Unsorted or includes invalid matches.	Riders need to find and pick rides
FR-004	Functional	The system shall calculate fuel cost split between driver and passengers based on shared distance and fuel rate.	High	Pass: Cost shown to both parties before confirmation. Fail: Wrong amount or not displayed.	Fair cost sharing is the main feature
FR-005	Functional	The system shall allow rider and driver to rate each other (1-5) after a trip and show aggregate rating on profiles.	Medium	Pass: Rating saved and aggregate updates. Fail: Rating not recorded.	Builds trust between users
NFR-001	Nonfunc.	User ratings, driving license documents, and travel histories must be stored with encryption and secure backups.	High	Pass: Security audit confirms encrypted storage. Fail: Any PII found unencrypted.	Data protection and user privacy
NFR-002	Nonfunc.	The system shall return ride match results within 3 seconds under 500 concurrent users.	High	Pass: 95th percentile latency < 3s in load test. Fail: Latency exceeds 3s.	Fast response needed during peak commute hours

2. UML Use-Case Diagram

Actors: Rider Commuter, Driver Host, Payment Gateway

Relationships: UC-03 includes UC-04 (route compatibility), Override detour tolerance extends UC-03



3. Use-Case Flow: UC-03 Request Ride Match

Primary Actor: Rider Commuter

Secondary Actors: Driver Host, Payment Gateway

Description: Rider finds a compatible driver, system checks route compatibility and calculates cost split.

Preconditions:

- Rider is logged in and has a saved commute route
- At least one driver with overlapping route exists

Postconditions:

- Ride is confirmed and both parties get a notification
- Fuel cost split is calculated and shown

Main Success Scenario

1. Rider taps "Find a Ride". System shows search form with saved route pre-filled.
2. Rider confirms route details and taps Search.
3. System runs route compatibility check (UC-04) for all active drivers within 1.5 km detour.
4. System shows list of matching drivers sorted by detour distance.
5. Rider picks a driver and taps "Request Ride".
6. System sends request to the Driver Host.
7. Driver accepts the ride request.
8. System calculates fuel cost split (UC-05) and shows it to both parties.
9. Rider confirms and pays through Payment Gateway.
10. System confirms booking, sends notifications to both, and logs the trip.

Alternate Flow

7a. Driver Declines

- 7a.1. System tells rider the driver declined.
- 7a.2. System shows remaining drivers from the list.
- 7a.3. Rider picks another driver, flow goes back to step 6.
- 7a.4. If no drivers left, system says "No rides found" and suggests trying later.

9a. Payment Fails

- 9a.1. Payment gateway returns failure.
- 9a.2. System shows error and asks rider to try another payment method.
- 9a.3. If payment fails again after 2 attempts, ride is cancelled and both are notified.